COMP3821/9801

Workshop Week 3

Greedy Algorithms on Graphs

Announcements.

Copyright Disclaimer. *Reproducing, publishing, posting, distributing, translating this document is an infringement of copyright and will be referred to UNSW Student Conduct and Integrity for action.*

Guided Problem

The demonstrators will solve the following problems. For the rest of the workshop, work in small groups to solve the remaining workshop exercises. There are optional problems at the end of this document for further practice.

Guided Problem

You have a collection of n lockboxes and m gold keys. Each key unlocks at most one box; however, each box might be unlocked by one key, by multiple keys, or by no keys at all. There are only two ways to open each box once it is locked: unlock it properly which requires having a matching key in your hand, or smash it to bits with a hammer.

Your baby brother, who loves playing with shiny objects, has somehow managed to lock all your keys inside the boxes! Luckily, your home security system recorded everything, so you know exactly which keys (if any) are inside each box and you can retrieve this information in constant-time; that is, you can assume that you know if box u contains a key to box v. You need to get all the keys back out of the boxes, because they are made of gold. Clearly, you have to smash at least one box.

Describe and analyse a polynomial-time algorithm that computes the smallest number of boxes that need to be smashed in order to retrieve all of the keys.

Guided Problem

Let $G = (V, E)$ be a directed graph. We say that G is *unilaterally connected* if, for every pair of distinct vertices $x, y \in V$, there exists a directed path from x to y or there exists a directed path from y to x, or there exist directed paths in both directions. Describe and analyse a polynomial-time algorithm that decides whether G is unilaterally connected.

Workshop Exercises

1. Let $G = (V, E)$ be a directed graph where each vertex $v \in V$ has an associated weight $w(v)$. Note that the weight may be positive, negative, or zero. A set $S \subseteq V$ *sees* the graph if, for each vertex $v \in V$, there exist a path from some vertex in S to v. The weight of set S is given by

$$w(S) = \sum_{v \in S} w(v).$$

   Describe and analyse a polynomial-time algorithm that computes the minimum weight set S that can see the graph G.

2. Let $G = (V, E)$ be a directed and connected graph. We say that G is *strongly connected* if every vertex in G can be reached by every other vertex in G. In other words, for every pair of distinct vertices $u, v \in V$, there exists a directed path from u to v and there exists a directed path from v to u.

Given a directed and connected graph $G = (V, E)$, describe and analyse a polynomial-time algorithm that computes the smallest number of edges from $(V \times V) \setminus E$ that needs to be added to G to make it strongly connected.

3. Let $G = (V, E)$ be a directed graph where each edge has been coloured either red or blue. A *red-blue path* is a path in sequence of distinct edges that alternate in colour, starting with a red edge and ending with a blue edge. We say that two distinct vertices $u, v \in V$ are *strongly red-blue connected* if there exist a red-blue path from u to v and there exist a red-blue path from v to u. Finally, a set of vertices C is a *strong red-blue component* if it is a maximal set of vertices such that every pair of vertices in C are strongly red-blue connected.

   Describe and analyse a polynomial-time algorithm that computes the strong red-blue components.

Optional Exercises

These exercises will not contain solutions. We encourage that you post on the forums to discuss the approaches for these problems. Additionally, you may find the hints helpful.

4. A directed graph G is said to be *strongly connected* if, for every pair $u, v \in V$ of distinct vertices, there exist a directed path from u to v. A vertex w is said to be *divisible* if there exist an integer k such that every loop containing w has length that is divisible by k. Prove that, if there exist a divisible vertex on a strongly-connected graph, then every vertex is divisible.

5. Let $G = (V, E)$ be a directed graph where a subset of vertices have been coloured either red or blue. We also know that there is at least one red vertex and there is at least one blue vertex. Let W denote a walk in G, and let r_W denote the number of times a red vertex was visited by W and let b_W denote the number of times a blue vertex was visited by W. We say that G is *vibing* if, for all $i, j \geq 0$, there exists a walk W such that $r_W \geq i$ and $b_W \geq j$.

   Describe and analyse a polynomial-time algorithm that decides if G is vibing.

6. Let $\Sigma = \{0, 1\}$. A *deterministic finite automaton* M on the alphabet Σ is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where

   - Q is a set of n states,
   - $\Sigma = \{0, 1\}$,
   - $\delta : Q \times \Sigma \to Q$ is a transition function that takes in a state and character from Σ as input and sends it to a state,
   - $q_0 \in Q$ is the *initial state*, and
   - $F \subseteq Q$ is a subset of *final states*.

   Let $w = a_0 a_1 \ldots a_k$ be a string, where $a_i \in \Sigma$. We say that M *accepts* w if there exist a sequence of states $s_0, \ldots, s_k$ such that

   - $s_0 = q_0$,
   - $s_{i+1} = \delta(s_i, a_{i+1})$ for $i = 0, \ldots, k-1$, and
   - $s_k \in F$.

   In other words, w is accepted by M if there exist a sequence of states that we can transition to, starting from the initial state q_0 and ending in one of the final states. Finally, the *language* $L(M)$ of M is the set of binary strings that is accepted by M. We can encode M with three arrays: $\delta_0[1, \ldots, n]$, $\delta_1[1, \ldots, n]$, and $F[1, \ldots, n]$. Here, for $k \in \{0, 1\}$, $\delta_k(i)$ is to be understood as

   if we are in state i and we read input k, then we transition to $\delta_k(j)$.

   Given a deterministic finite automaton M on n states, describe and analyse a polynomial-time algorithm that decides if the language $L(M)$ is infinite in cardinality; that is, you want to design an algorithm that determines whether M accepts infinitely many binary strings.

7. Let $G = (V, E)$ be a complete and weighted graph. Suppose that all of the edge weights are positive and unique. You are also given the minimum spanning tree T for G. You are given a new vertex v_{n+1} and the weights $w(v_{n+1}, v_j)$ between the

new vertex v_{n+1} and every other vertex in G. Describe and analyse a polynomial-time algorithm that produces a minimum spanning tree T' for the new graph containing the additional vertex.

8. Let $G = (V, E)$ be a connected, undirected, and weighted graph. Describe and analyse a polynomial-time algorithm that computes a spanning tree such that the largest weight among all of the edges in the spanning tree is minimised.

9. Let $G = (V, E)$ be an undirected and weighted graph, where $w(e) > 0$ is an injective weight function (i.e. all edge weights are unique). You are given that a particular edge $e \in E$ has its weight modified, and are asked to find the new minimum spanning tree. Of course, you *could* just recompute the minimum spanning tree but that's boring! We want to do something more clever. In all four cases, assume you are given the actual minimum spanning tree T in advance.

   (a) Suppose that the edge $e \in T$ has its weight decreased. Explain why T is still the minimum spanning tree.

   (b) Suppose that the edge $e \notin T$ has its weight increased. Explain why T is still the minimum spanning tree.

   (c) Suppose that the edge $e \in T$ has its weight increased. Describe and analyse an $O(|V| + |E|)$ algorithm to compute the new minimum spanning tree.

   (d) Suppose that the edge $e \notin T$ has its weight decreased. Describe and analyse an $O(|E|)$ algorithm to compute the new minimum spanning tree.

10. Assume that you are given a complete and undirected graph $G = (V, E)$ with edge weights $\{w(e) : e \in E\}$ and a proper subset of vertices $U \subset V$. Describe and analyse a polynomial-time algorithm that produces the minimum spanning tree of G in which all of the vertices in U are leaves.

11. Let $G = (V, E)$ be a weighted graph. Describe and analyse a polynomial-time algorithm that finds all edges that appear in *every* minimum spanning tree.

12. Let $G = (V, E)$ be an undirected graph. A *bridge* edge is an edge whose deletion increases the number of connected components in G. A *bridgeless graph* is a graph with no bridge edges. Finally, a *strong orientation* assigns each edge of G a direction such that G becomes a strongly connected graph.

    Robbins' theorem says that a connected graph G has a strong orientation if and only if G is bridgeless. Therefore, we will only look at bridgeless and connected graphs. Let $G = (V, E)$ be a bridgeless and connected graph. Describe and analyse linear-time algorithm to find a strong orientation of G.

    Note. *A similar algorithm can be used to find ear decompositions.*

Hints

4. Suppose that u is divisible and let v be some arbitrary vertex. You can construct a path from u to v and a path from v to u since G is strongly-connected; how do you conclude that v must be divisible too?
5. What happens if a strongly-connected component contains a red and blue vertex? What happens if a strongly-connected component does not contain a red or blue vertex?
6. Build the graph corresponding to the automaton M. If a strongly-connected component containing a final vertex is reachable from the start vertex, what does that imply?
7. The new spanning tree must consist of which edges?
8. Why is the minimum spanning tree optimal?
9. For part (a), is this obvious? For part (b), the argument is simplified because all of the edge weights are unique! For part (d), either e belongs in the new MST or it doesn't.
10. Consider the graph $G - U$ by deleting all vertices in U and then add on edges one at a time that connect to vertices in U.
11. Start by thinking about how to test if there exist an MST that does not contain e? How many such edges do we need to test?
12. Think about a DFS tree.